

透過 WebRTC 即時通訊

來源：<https://codelabs.developers.google.com/codelabs/webrtc-web?hl=zh-tw>

步驟數：10

瞭解如何在兩個瀏覽器之間串流媒體和資料。熟悉 WebRTC 的核心 API 和技術。使用 getUserMedia、CSS 和 Canvas 元素擷取及操控圖片。使用資料管道設定對等互連連線，並在瀏覽器之間直接交換資料。最後，使用 Node.js 設定信號伺服器。

目錄

1. [1. 簡介](#)
2. [2. 總覽](#)
3. [3. 取得範例程式碼](#)
4. [4. 從網路攝影機串流影片](#)
5. [5. 使用 RTCPeerConnection 串流播放影片](#)
6. [6. 使用 RTCDataChannel 交換資料](#)
7. [7. 設定信號服務來交換訊息](#)
8. [8. 合併對等互連和信號](#)
9. [9. 拍照並透過資料管道分享相片](#)
10. [10. 恭喜](#)

1. 簡介

WebRTC 是一項開放原始碼專案，可讓網頁和原生應用程式即時傳輸音訊、視訊和資料。

WebRTC 包含多個 JavaScript API，點選連結即可查看示範。

- `getUserMedia()`：擷取音訊和視訊。
- `MediaRecorder`：錄製音訊和視訊。
- `RTCPeerConnection`：在使用者之間串流音訊和視訊。
- `RTCDataChannel`：在使用者之間串流資料。

我可以在哪裡使用 WebRTC？

Firefox、Opera，以及 Chrome 電腦版和 Android 版。WebRTC 也適用於 iOS 和 Android 上的原生應用程式。

什麼是信號？

WebRTC 會使用 `RTCPeerConnection` 在瀏覽器之間傳輸串流資料，但同時也需要協調通訊及傳送控制訊息的機制，這個程序稱為信號傳輸。WebRTC 並未指定信號傳輸方法和通訊協定，在本程式碼研究室中，您將使用 Socket.IO 傳送訊息，但還有許多替代方案。

什麼是 STUN 和 TURN？

WebRTC 的設計是點對點運作，因此使用者可以透過最直接的路徑連線。不過，WebRTC 的設計宗旨是因應現實世界的網路環境：用戶端應用程式必須遍歷 NAT 閘道和防火牆，且點對點網路必須提供備援機制，以防直接連線失敗。在這個過程中，WebRTC API 會使用 STUN 伺服器取得電腦的 IP 位址，並使用 TURN 伺服器做為中繼伺服器，以防對等互連通訊失敗。(如要瞭解詳情，請參閱「[現實世界中的 WebRTC](#)」一文)。

WebRTC 安全嗎？

所有 WebRTC 元件都必須加密，且 JavaScript API 只能從安全來源 (HTTPS 或 localhost) 使用。WebRTC 標準未定義信號機制，因此請務必使用安全通訊協定。

想瞭解更多資訊嗎？歡迎前往 webrtc.org 查看相關資源。

2. 總覽

建構應用程式，透過網路攝影機取得影片和拍攝快照，並透過 WebRTC 以對等互連方式分享。過程中，您將瞭解如何使用核心 WebRTC API，以及如何使用 Node.js 設定訊息伺服器。

課程內容

- 取得網路攝影機的影像
- 使用 `RTCPeerConnection` 串流播放影片
- 使用 `RTCDataChannel` 串流資料
- 設定信號服務來交換訊息
- 合併對等互連和信號
- 拍照並透過資料管道分享相片

軟硬體需求

- Chrome 47 以上版本
 - [Chrome 的網路伺服器](#)，或使用您選擇的網路伺服器。
 - 程式碼範例
 - 文字編輯器
 - 具備 HTML、CSS 和 JavaScript 的基本知識
-

步驟 3 · 約 2 分鐘

3. 取得範例程式碼

下載程式碼

如果您熟悉 Git，可以複製 GitHub 上的程式碼，下載本程式碼研究室的程式碼：

```
git clone https://github.com/googlecode/adaptive-web-media
```

或者，您也可以點選下列按鈕，下載程式碼的 .zip 檔案：

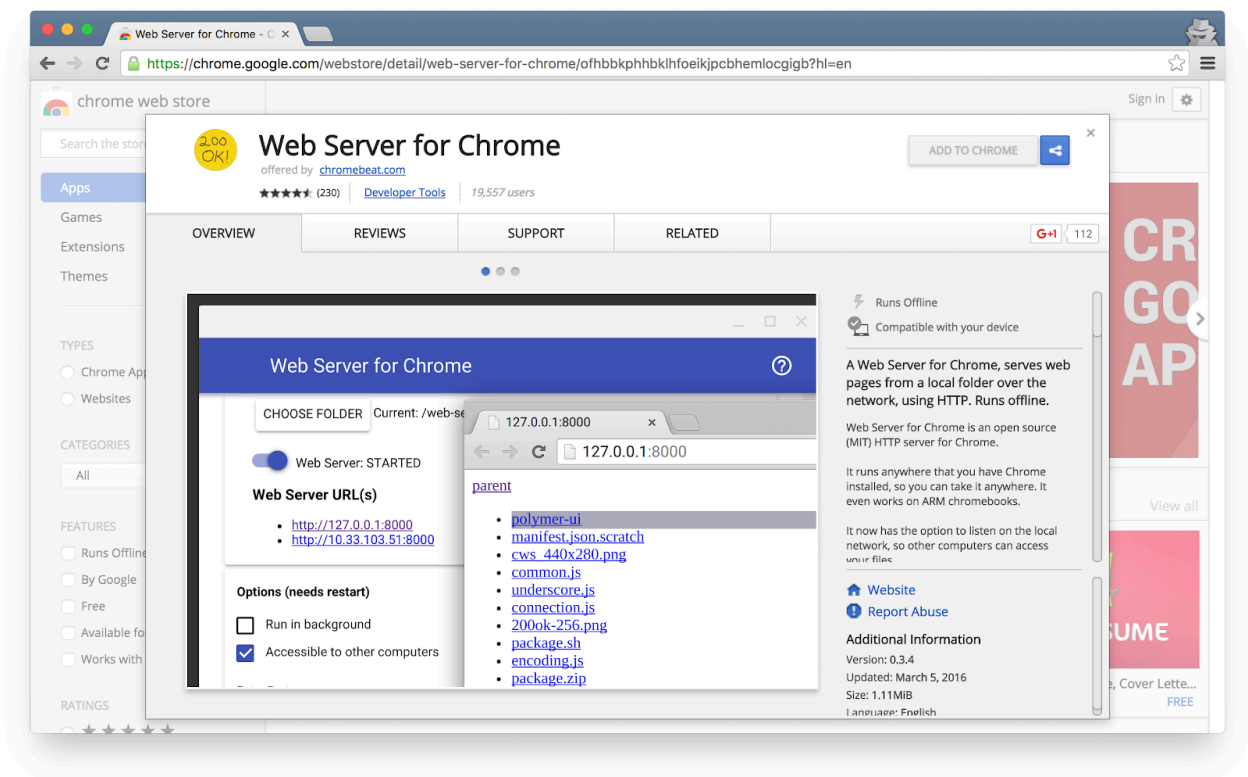
開啟下載的 ZIP 檔案，這會解壓縮專案資料夾 (**adaptive-web-media**)，其中包含本程式碼研究室每個步驟的資料夾，以及您需要的所有資源。

您會在名為 **work** 的目錄中完成所有程式碼編寫工作。

step-nn 資料夾包含本程式碼研究室每個步驟的完成版本。僅供參考。

安裝並驗證網路伺服器

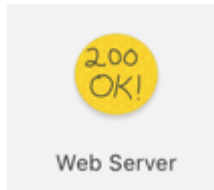
您可以自由使用自己的網路伺服器，但本程式碼研究室的設計可與 Chrome 網路伺服器完美搭配。如果尚未安裝該應用程式，可以從 Chrome 線上應用程式商店安裝。



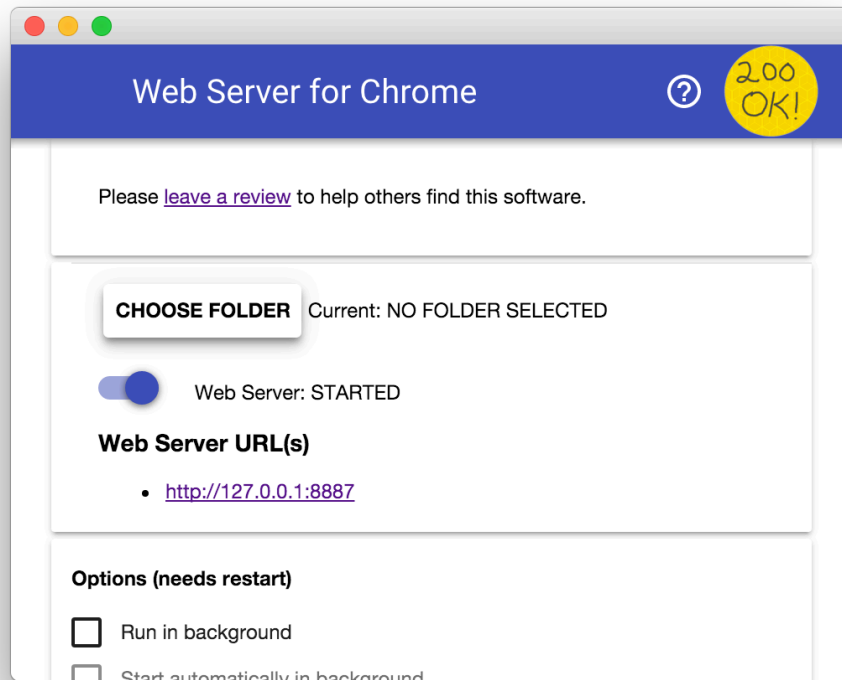
安裝 **Web Server for Chrome** 應用程式後，按一下書籤列、新分頁或應用程式啟動器中的 Chrome 應用程式捷徑：



按一下「Web Server」圖示：



接著，您會看到這個對話方塊，可供您設定本機網路伺服器：



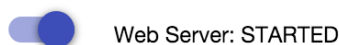
按一下「選擇資料夾」按鈕，然後選取剛建立的「work」資料夾。這樣一來，您就能透過「Web Server URL(s)」部分「Web Server」對話方塊中醒目顯示的網址，在 Chrome 中查看進行中的工作。

在「選項」下方，勾選「Automatically show index.html」(自動顯示 index.html) 旁邊的方塊，如下所示：

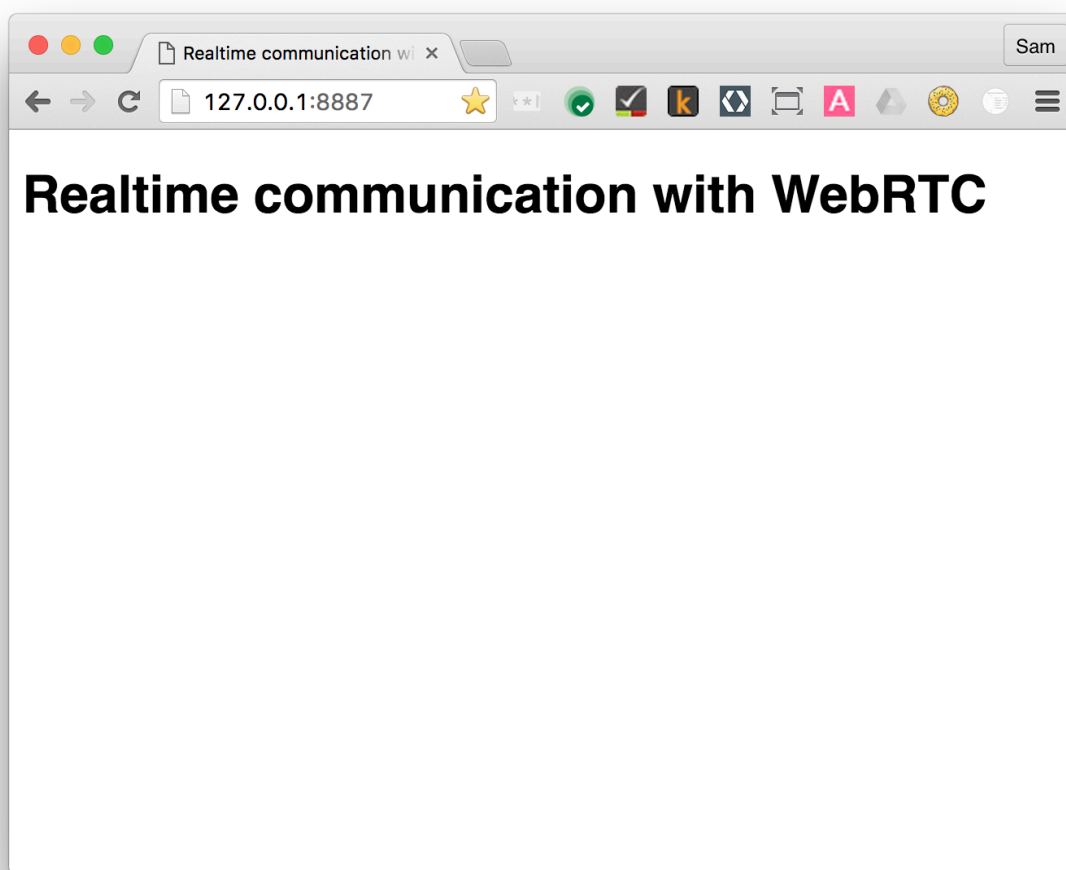
Options (needs restart)

- ☐ Run in background
- ☐ Start automatically in background
- ☐ Accessible to other computers
- ☒ Automatically show index.html

接著，將標示為「Web Server: STARTED」(網路伺服器：已啟動) 的切換鈕滑動至左側，然後再滑動至右側，即可停止並重新啟動伺服器。



現在點選醒目顯示的 Web Server URL，在網路瀏覽器中造訪工作網站。您應該會看到類似下方的頁面，對應於 **work/index.html**：



顯然，這個應用程式還不會執行任何有趣的操作，目前只是用來確保網路伺服器正常運作的最小骨架。您會在後續步驟中新增功能和版面配置功能。

從現在開始，所有測試和驗證作業都應使用這個網路伺服器設定執行。通常只要重新整理測試瀏覽器分頁即可。

步驟 4 · 約 5 分鐘

4. 從網路攝影機串流影片

課程內容

這個步驟會說明如何：

- 從網路攝影機取得影片串流。
- 操控串流播放。
- 使用 CSS 和 SVG 操控影片。

這個步驟的完整版本位於「step-01」**step-01**資料夾中。

一點點 HTML...

在 **work** 目錄的 **index.html** 中新增 `video` 元素和 `script` 元素：

```
<!DOCTYPE html>
<html>

<head>

  <title>Realtime communication with WebRTC</title>

  <link rel="stylesheet" href="css/main.css" />

</head>

<body>

  <h1>Realtime communication with WebRTC</h1>

  <video autoplay playsinline></video>

  <script src="js/main.js"></script>

</body>

</html>
```

...以及少許 JavaScript

在 **js** 資料夾的 **main.js** 中新增下列內容：


```
'use strict';

// On this codelab, you will be streaming only video (video: true).
const mediaStreamConstraints = {
  video: true,
};

// Video element where stream will be placed.
const localVideo = document.querySelector('video');

// Local stream that will be reproduced on the video.
let localStream;

// Handles success by adding the MediaStream to the video element.
function gotLocalMediaStream(mediaStream) {
  localStream = mediaStream;
  localVideo.srcObject = mediaStream;
}

// Handles error by logging a message to the console with the error message.
function handleLocalMediaStreamError(error) {
  console.log('navigator.getUserMedia error: ', error);
}

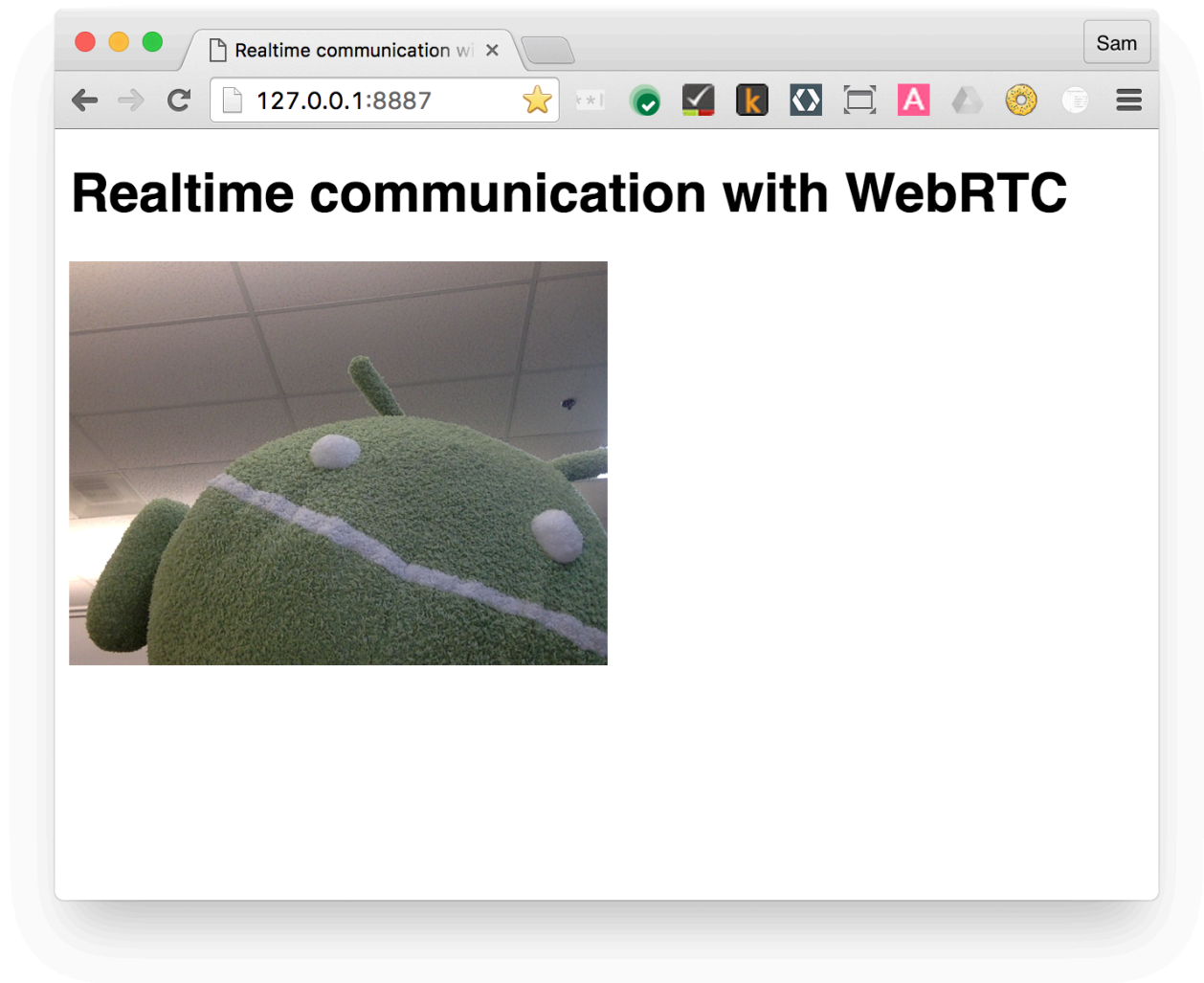
// Initializes media stream.
navigator.mediaDevices.getUserMedia(mediaStreamConstraints)
  .then(gotLocalMediaStream).catch(handleLocalMediaStreamError);
```

這裡的所有 JavaScript 範例都使用 `'use strict';`，避免常見的程式碼陷阱。

如要進一步瞭解這項變更的意義，請參閱「[ECMAScript 5 Strict Mode, JSON, and More](#)」。

立即試用

在瀏覽器中開啟 **index.html**，您應該會看到類似下列的畫面 (當然，畫面會顯示網路攝影機的影像！)：



運作方式

在 `getUserMedia()` 呼叫後，瀏覽器會向使用者要求存取相機的權限 (如果這是目前來源首次要求存取相機)。如果成功，系統會傳回 [MediaStream](#)，媒體元素可透過 `srcObject` 屬性使用該串流：

```
navigator.mediaDevices.getUserMedia(mediaStreamConstraints)
    .then(gotLocalMediaStream).catch(handleLocalMediaStreamError);

}
```

```
function gotLocalMediaStream(mediaStream) {
    localVideo.srcObject = mediaStream;
}
```

`constraints` 引數可讓您指定要取得的媒體。在本例中，由於音訊預設為停用，因此只會顯示影片：

```
const mediaStreamConstraints = {
    video: true,
};
```

您可以針對其他需求 (例如影片解析度) 使用限制：

```
const hdConstraints = {
  video: {
    width: {
      min: 1280
    },
    height: {
      min: 720
    }
  }
}
```

[MediaTrackConstraints 規格](#)列出所有可能的限制類型，但並非所有瀏覽器都支援所有選項。如果目前所選攝影機不支援要求的解析度，系統會拒絕 `getUserMedia()` 並顯示 `OverconstrainedError`，且不會提示使用者授予攝影機存取權。

如要查看如何使用限制條件要求不同解析度的示範，請按[這裡](#)；如要查看如何使用限制條件選擇攝影機和麥克風的示範，請按[這裡](#)。

如果 `getUserMedia()` 成功，網路攝影機的影像串流會設為影片元素的來源：

```
function gotLocalMediaStream(mediaStream) {
  localVideo.srcObject = mediaStream;
}
```

獎勵分數

- 傳遞至 `getUserMedia()` 的 `localStream` 物件位於全域範圍，因此您可以從瀏覽器控制台檢查該物件：開啟控制台，輸入 `stream` 並按下 Return 鍵。(如要在 Chrome 中查看控制台，請按下 Ctrl+Shift+J 鍵，或在 Mac 上按下 Command+Option+J 鍵)。
- `localStream.getVideoTracks()` 會傳回什麼？
- 請嘗試撥打 `localStream.getVideoTracks()[0].stop()`。
- 查看限制物件：將其變更為 `{audio: true, video: true}` 時會發生什麼情況？
- 影片元素的大小為何？如何從 JavaScript 取得影片的自然大小 (而非顯示大小)？使用 Chrome 開發人員工具檢查。
- 請嘗試為影片元素新增 CSS 濾鏡。例如：

```
video {
  filter: blur(4px) invert(1) opacity(0.5);
}
```

- 建議您新增 SVG 濾鏡。例如：

```
video {
  filter: hue-rotate(180deg) saturate(200%);
}
```

您學到的內容

您在此步驟中瞭解到如何：

- 從網路攝影機取得影片。
- 設定媒體限制。
- 干擾影片元素。

這個步驟的完整版本位於「step-01」 **step-01** 資料夾中。

訣竅

- 別忘了 `video` 元素上的 `autoplay` 屬性。否則只會看到單一影格！
- `getUserMedia()` 限制還有許多其他選項。請前往 webrtc.github.io/samples/src/content/peerconnection/constraints 查看示範。如您所見，該網站上有許多有趣的 WebRTC 範例。

最佳做法

- 請確認影片元素不會溢出容器。我們新增了 `width` 和 `max-width`，可設定影片的偏好大小和最大大小。瀏覽器會自動計算高度：

```
video {  
  max-width: 100%;  
  width: 320px;  
}
```

下一步

你已經有影片，但該如何串流播放？請參閱下一個步驟！

5. 使用 RTCPeerConnection 串流播放影片

課程內容

這個步驟會說明如何：

- 使用 WebRTC 墊片 [adapter.js](#)，抽象化瀏覽器差異。
- 使用 RTCPeerConnection API 串流播放影片。
- 控制媒體擷取和串流。

這個步驟的完整版本位於「step-2」**step-2**資料夾中。

什麼是 RTCPeerConnection？

RTCPeerConnection 是一項 API，可發出 WebRTC 呼叫來串流影片和音訊，並交換資料。

這個範例會在同一網頁上，設定兩個 RTCPeerConnection 物件 (稱為對等互連) 之間的連線。

實用性不高，但有助於瞭解 RTCPeerConnection 的運作方式。

新增影片元素和控制按鈕

在 **index.html** 中，將單一影片元素替換為兩個影片元素和三個按鈕：

```
<video id="localVideo" autoplay playsinline></video>
<video id="remoteVideo" autoplay playsinline></video>

<div>
  <button id="startButton">Start</button>
  <button id="callButton">Call</button>
  <button id="hangupButton">Hang Up</button>
</div>
```

一個影片元素會顯示來自 `getUserMedia()` 的串流，另一個則會顯示透過 RTCPeerconnection 串流的相同影片。(在實際應用中，一個影片元素會顯示本機串流，另一個則顯示遠端串流)。

新增 adapter.js shim

在 **main.js** 的連結上方，新增指向目前 [adapter.js](#) 版本的連結：

```
<script src="https://webrtc.github.io/adapter/adapter-latest.js"></script>
```

adapter.js 是用來保護應用程式不受規格變更和前置字串差異影響的墊片。(但事實上，WebRTC 實作項目使用的標準和通訊協定非常穩定，且只有少數前置字元名稱)。

在這個步驟中，我們已連結至最新版的 **adapter.js**，這對程式碼研究室來說沒問題，但可能不適合用於正式版應用程式。[adapter.js GitHub 存放區](#)說明如何確保應用程式一律存取最新版本。

若要進一步瞭解 WebRTC 互通性，請參閱 [WebRTC 的互通性注意事項](#)。

Index.html 現在應如下所示：

```
<!DOCTYPE html>
<html>

<head>
  <title>Realtime communication with WebRTC</title>
  <link rel="stylesheet" href="css/main.css" />
</head>

<body>
  <h1>Realtime communication with WebRTC</h1>

  <video id="localVideo" autoplay playsinline></video>
  <video id="remoteVideo" autoplay playsinline></video>

  <div>
    <button id="startButton">Start</button>
    <button id="callButton">Call</button>
    <button id="hangupButton">Hang Up</button>
  </div>

  <script src="https://webrtc.github.io/adapter/adapter-latest.js"></script>
  <script src="js/main.js"></script>
</body>
</html>
```

安裝 RTCPeerConnection 程式碼

將 **main.js** 替換為 **step-02** 資料夾中的版本。

在程式碼實驗室中，以剪下及貼上方式處理大量程式碼並非理想做法，但為了讓 RTCPeerConnection 正常運作，我們別無選擇，只能全力以赴。

稍後會說明程式碼的運作方式。

撥打電話

開啟 **index.html**，按一下「Start」按鈕，從網路攝影機取得影片，然後按一下「Call」建立對等互連連線。兩個影片元素中應會顯示相同的影片 (來自網路攝影機)。查看瀏覽器控制台，瞭解 WebRTC 記錄。

運作方式

這個步驟會執行許多作業...

如果不想略過下方的說明，也沒關係。

您還是可以繼續進程式碼研究室！

WebRTC 會使用 `RTCPeerConnection` API 設定連線，在 WebRTC 用戶端 (又稱對等互連) 之間串流影片。

在本例中，兩個 `RTCPeerConnection` 物件位於同一頁面：`pc1` 和 `pc2`。實用性不高，但很適合用來示範 API 的運作方式。

在 WebRTC 對等互連之間設定通話時，需要完成三項工作：

- 為通話的每一端建立 `RTCPeerConnection`，並在每一端新增 `getUserMedia()` 的本機串流。
- 取得及分享網路資訊：潛在連線端點稱為 ICE 候選項目。
- 取得及分享本機和遠端說明：本機媒體的中繼資料，格式為 SDP。

假設 Alice 和 Bob 想使用 `RTCPeerConnection` 設定視訊通訊。

首先，愛麗絲和鮑伯會交換網路資訊。「尋找候選項目」是指使用 ICE 架構尋找網路介面和通訊埠的程序。

1. Alice 使用 `onicecandidate` (`addEventListener('icecandidate')`) 處理常式建立 `RTCPeerConnection` 物件。這對應於 `main.js` 中的下列程式碼：

```
let localPeerConnection;
```

```
localPeerConnection = new RTCPeerConnection(servers);
localPeerConnection.addEventListener('icecandidate', handleConnection);
localPeerConnection.addEventListener(
  'iceconnectionstatechange', handleConnectionChange);
```

本範例不會使用 `RTCPeerConnection` 的 `servers` 引數。

您可以在這裡指定 STUN 和 TURN 伺服器。

WebRTC 的設計是點對點運作，因此使用者可以透過最直接的路徑連線。不過，WebRTC 的設計宗旨是因應現實世界的網路環境：用戶端應用程式必須遍歷 NAT 閘道和防火牆，且點對點網路必須提供備援機制，以防直接連線失敗。

在這個過程中，WebRTC API 會使用 STUN 伺服器取得電腦的 IP 位址，並使用 TURN 伺服器做為中繼伺服器，以防對等互連通訊失敗。詳情請參閱「[現實世界中的 WebRTC](#)」。

2. Alice 呼叫 `getUserMedia()`，並新增傳遞至該項目的串流：

```
navigator.mediaDevices.getUserMedia(mediaStreamConstraints).
  then(getLocalMediaStream).
  catch(handleLocalMediaStreamError);
```

```
function gotLocalMediaStream(mediaStream) {
  localVideo.srcObject = mediaStream;
  localStream = mediaStream;
  trace('Received local stream.');
```

```
  callButton.disabled = false; // Enable call button.
}
```

```
localPeerConnection.addStream(localStream);
trace('Added local stream to localPeerConnection.');
```

3. 網路候選項目可用時，系統會呼叫步驟 1 中的 `onIceCandidate` 處理常式。
4. Alice 將序列化候選人資料傳送給 Bob。在實際應用程式中，這個程序 (稱為「信號傳輸」) 會透過訊息服務進行，我們會在後續步驟中說明如何操作。當然，在這個步驟中，兩個 `RTCPeerConnection` 物件位於同一網頁，可以直接通訊，不需要外部訊息傳遞。
5. 當 Bob 收到 Alice 的候選訊息時，他會呼叫 `addIceCandidate()`，將候選項目新增至遠端對等互連說明：

```
function handleConnection(event) {
  const peerConnection = event.target;
  const iceCandidate = event.candidate;

  if (iceCandidate) {
    const newIceCandidate = new RTCIceCandidate(iceCandidate);
    const otherPeer = getOtherPeer(peerConnection);

    otherPeer.addIceCandidate(newIceCandidate)
      .then(() => {
        handleConnectionSuccess(peerConnection);
      }).catch((error) => {
        handleConnectionFailure(peerConnection, error);
      });

    trace(`${getPeerName(peerConnection)} ICE candidate:\n` +
      `${event.candidate.candidate}.`);
  }
}
```

WebRTC 對等互連也需要找出並交換本機和遠端音訊與視訊媒體資訊，例如解析度和轉碼器功能。如要交換媒體設定資訊，請使用「工作階段描述通訊協定」(Session Description Protocol，簡稱 [SDP](#)) 格式，交換中繼資料 Blob，也就是「提案」和「回覆」：

1. 小莉執行 `RTCPeerConnection createOffer()` 方法。傳回的 Promise 會提供 `RTCSessionDescription`：Alice 的本機工作階段說明：

```
trace('localPeerConnection createOffer start.');
```

```
localPeerConnection.createOffer(offerOptions)
  .then(createdOffer).catch(setSessionDescriptionError);
```

2. 如果成功，Alice 會使用 `setLocalDescription()` 設定本機說明，然後透過信號傳輸管道將工作階段說明傳送給 Bob。
3. Bob 使用 `setRemoteDescription()` 將 Alice 傳送給他的說明設為遠端說明。
4. Bob 執行 `RTCPeerConnection createAnswer()` 方法，並將從 Alice 取得的遠端說明傳遞給該方法，以便產生與 Alice 說明相容的本機工作階段。`createAnswer()` promise 會傳遞 `RTCSessionDescription`：Bob 會將其設為本機說明，並傳送給 Alice。
5. 小莉收到志明的會期說明後，會使用 `setRemoteDescription()` 將其設為遠端說明。


```
// Logs offer creation and sets peer connection session descriptions.
function createdOffer(description) {
  trace(`Offer from localPeerConnection:\n${description.sdp}`);

  trace('localPeerConnection setLocalDescription start.');
```

```
localPeerConnection.setLocalDescription(description)
  .then(() => {
    setLocalDescriptionSuccess(localPeerConnection);
  }).catch(setSessionDescriptionError);

  trace('remotePeerConnection setRemoteDescription start.');
```

```
remotePeerConnection.setRemoteDescription(description)
  .then(() => {
    setRemoteDescriptionSuccess(remotePeerConnection);
  }).catch(setSessionDescriptionError);

  trace('remotePeerConnection createAnswer start.');
```

```
remotePeerConnection.createAnswer()
  .then(createdAnswer)
  .catch(setSessionDescriptionError);
}

// Logs answer to offer creation and sets peer connection session descriptions.
function createdAnswer(description) {
  trace(`Answer from remotePeerConnection:\n${description.sdp}.`);

  trace('remotePeerConnection setLocalDescription start.');
```

```
remotePeerConnection.setLocalDescription(description)
  .then(() => {
    setLocalDescriptionSuccess(remotePeerConnection);
  }).catch(setSessionDescriptionError);

  trace('localPeerConnection setRemoteDescription start.');
```

```
localPeerConnection.setRemoteDescription(description)
  .then(() => {
    setRemoteDescriptionSuccess(localPeerConnection);
  }).catch(setSessionDescriptionError);
}
```

6. Ping!

獎勵分數

1. 請查看 **chrome://webrtc-internals**。這會提供 WebRTC 統計資料和偵錯資料。(如需完整的 Chrome 網址清單，請前往 **chrome://about**。)
2. 使用 CSS 設定網頁樣式：
 - 將影片並排顯示。
 - 將按鈕設為相同寬度，並放大文字。
 - 確認版面配置在行動裝置上正常運作。

3. 在 Chrome 開發人員工具主控台中，查看 `localStream`、`localPeerConnection` 和 `remotePeerConnection`。
4. 在控制台中查看 `localPeerConnectionpc1.localDescription`。SDP 格式為何？

您學到的內容

您在此步驟中瞭解到如何：

- 使用 WebRTC 墊片 [adapter.js](#)，抽象化瀏覽器差異。
- 使用 `RTCPeerConnection` API 串流播放影片。
- 控制媒體擷取和串流。
- 在對等互連裝置之間分享媒體和網路資訊，以便進行 WebRTC 通話。

這個步驟的完整版本位於「step-2」**step-2**資料夾中。

訣竅

- 這個步驟的內容非常豐富，如要尋找其他資源，進一步瞭解 `RTCPeerConnection`，請參閱 webrtc.org。如果您想使用 WebRTC，但不想處理 API，本頁面提供 JavaScript 架構建議。
- 如要進一步瞭解 `adapter.js` shim，請前往 [adapter.js GitHub 存放區](#)。
- 想看看全球最佳視訊通訊應用程式的樣貌嗎？請參閱 AppRTC，這是 WebRTC 專案的標準應用程式，用於 WebRTC 通話：[應用程式](#)、[程式碼](#)。通話設定時間少於 500 毫秒。

最佳做法

- 為確保程式碼能因應未來變化，請使用新的 Promise 型 API，並透過 [adapter.js](#) 啟用與不支援這些 API 的瀏覽器相容性。

下一步

這個步驟說明如何使用 WebRTC 在對等互連裝置間串流播放影片，但本程式碼實驗室也與資料有關！

在下一個步驟中，瞭解如何使用 `RTCDataChannel` 串流任意資料。

步驟 6 · 約 5 分鐘

6. 使用 RTCDataChannel 交換資料

課程內容

- 如何在 WebRTC 端點 (對等互連) 之間交換資料。

這個步驟的完整版本位於「step-03」**step-03**資料夾中。

更新 HTML

在這個步驟中，您會使用 WebRTC 資料通道，在同一網頁上的兩個 `textarea` 元素之間傳送文字。這並非十分實用，但可說明如何使用 WebRTC 分享資料和串流影片。

從 **index.html** 移除影片和按鈕元素，並替換為下列 HTML：

```
<textarea id="dataChannelSend" disabled
  placeholder="Press Start, enter some text, then press Send."></textarea>
<textarea id="dataChannelReceive" disabled></textarea>

<div id="buttons">
  <button id="startButton">Start</button>
  <button id="sendButton">Send</button>
  <button id="closeButton">Stop</button>
</div>
```

一個文字區用於輸入文字，另一個則會顯示對等互傳的文字。

index.html 現在應如下所示：

```

<!DOCTYPE html>
<html>

<head>

  <title>Realtime communication with WebRTC</title>

  <link rel="stylesheet" href="css/main.css" />

</head>

<body>

  <h1>Realtime communication with WebRTC</h1>

  <textarea id="dataChannelSend" disabled
    placeholder="Press Start, enter some text, then press Send."></textarea>
  <textarea id="dataChannelReceive" disabled></textarea>

  <div id="buttons">
    <button id="startButton">Start</button>
    <button id="sendButton">Send</button>
    <button id="closeButton">Stop</button>
  </div>

  <script src="https://webrtc.github.io/adapter/adapter-latest.js"></script>
  <script src="js/main.js"></script>

</body>

</html>

```

更新 JavaScript

將 **main.js** 替換為 **step-03/js/main.js** 的內容。

與上一個步驟相同，在程式碼研究室中對大量程式碼進行剪下及貼上作業並非理想做法，但 (與 `RTCPeerConnection` 相同) 沒有其他替代方案。

試試對等互傳串流資料：開啟 **index.html**，按下 **Start** 設定對等連線，在左側的 `textarea` 中輸入一些文字，然後按一下 **Send**，透過 WebRTC 資料通道傳輸文字。

運作方式

這段程式碼使用 `RTCPeerConnection` 和 `RTCDataChannel`，啟用文字訊息交換功能。

這個步驟中的大部分程式碼都與 `RTCPeerConnection` 範例相同。

`sendData()` 和 `createConnection()` 函式包含大部分的新程式碼：

```

function createConnection() {
  dataChannelSend.placeholder = '';
  var servers = null;
  pcConstraint = null;
  dataConstraint = null;
  trace('Using SCTP based data channels');
  // For SCTP, reliable and ordered delivery is true by default.
  // Add localConnection to global scope to make it visible
  // from the browser console.
  window.localConnection = localConnection =
    new RTCPeerConnection(servers, pcConstraint);
  trace('Created local peer connection object localConnection');

  sendChannel = localConnection.createDataChannel('sendDataChannel',
    dataConstraint);
  trace('Created send data channel');

  localConnection.onicecandidate = iceCallback1;
  sendChannel.onopen = onSendChannelStateChange;
  sendChannel.onclose = onSendChannelStateChange;

  // Add remoteConnection to global scope to make it visible
  // from the browser console.
  window.remoteConnection = remoteConnection =
    new RTCPeerConnection(servers, pcConstraint);
  trace('Created remote peer connection object remoteConnection');

  remoteConnection.onicecandidate = iceCallback2;
  remoteConnection.ondatachannel = receiveChannelCallback;

  localConnection.createOffer().then(
    gotDescription1,
    onCreateSessionDescriptionError
  );
  startButton.disabled = true;
  closeButton.disabled = false;
}

function sendData() {
  var data = dataChannelSend.value;
  sendChannel.send(data);
  trace('Sent Data: ' + data);
}

```

RTCDataChannel 的語法刻意與 WebSocket 相似，包含 `send()` 方法和 `message` 事件。

請注意 `dataConstraint` 的用法。您可以設定資料管道，啟用不同類型的資料共用功能，例如優先確保資料傳送的可靠性，而非效能。如要進一步瞭解選項，請前往 [Mozilla 開發人員網路](#)。

三種限制

這太令人困惑了！

不同類型的 WebRTC 通話設定選項通常都稱為「限制」。

進一步瞭解限制和選項：

- [RTCPeerConnection](#)
- [RTCDataChannel](#)
- [getUserMedia\(\)](#)

獎勵分數

1. WebRTC 資料通道使用的通訊協定 [SCTP](#) 預設會可靠地依序傳送資料。RTCDataChannel 何時需要提供可靠的資料傳輸服務？何時效能更重要 (即使這表示會遺失部分資料)？
2. 使用 CSS 改善網頁版面配置，並在「dataChannelReceive」文字區域中新增預留位置屬性。
3. 在行動裝置上測試網頁。

您學到的內容

您在此步驟中瞭解到如何：

- 在兩個 WebRTC 對等互連裝置之間建立連線。
- 在對等互連裝置之間交換文字資料。

這個步驟的完整版本位於「step-03」**step-03**資料夾中。

瞭解詳情

- [WebRTC 資料管道](#) (幾年前的文章，但仍值得一讀)
- [為什麼 WebRTC 的資料管道選用 SCTP？](#)

下一步

您已瞭解如何在同一網頁上的對等互連裝置之間交換資料，但如何在不同機器之間交換資料呢？首先，您需要設定信號傳輸管道，交換中繼資料訊息。請參閱下一個步驟瞭解詳情！

7. 設定信號服務來交換訊息

課程內容

在本步驟中，您將瞭解如何：

- 使用 `npm` 安裝 `package.json` 中指定的專案依附元件
- 執行 Node.js 伺服器，並使用 `node-static` 提供靜態檔案。
- 使用 Socket.IO 在 Node.js 上設定訊息服務。
- 並用來建立「聊天室」和交換訊息。

這個步驟的完整版本位於「step-04」**step-04**資料夾中。

概念

如要設定及維護 WebRTC 通話，WebRTC 用戶端 (對等互連) 必須交換中繼資料：

- 候選人 (電視網) 資訊。
- **Offer** 和 **answer** 訊息，提供媒體相關資訊，例如解析度和轉碼器。

換句話說，必須先交換中繼資料，才能進行音訊、視訊或資料的對等串流。這項程序稱為「信號傳輸」。

在先前的步驟中，傳送者和接收者的 `RTCPeerConnection` 物件位於同一網頁上，因此「信號」只是在物件之間傳遞中繼資料。

在實際應用程式中，傳送者和接收者 `RTCPeerConnection` 會在不同裝置的網頁上執行，您需要讓兩者傳輸中繼資料。

為此，您需要使用**信號伺服器**：這類伺服器可在 WebRTC 用戶端 (對等互連) 之間傳遞訊息。實際訊息是純文字：字串化的 JavaScript 物件。

必要條件：安裝 Node.js

如要執行本程式碼研究室的後續步驟 (資料夾 **step-04** 到 **step-06**)，您需要使用 Node.js 在 localhost 上執行伺服器。

您可以從[這個連結](#)下載並安裝 Node.js，或透過偏好的**套件管理工具**安裝。

安裝完成後，您就能匯入後續步驟 (執行 `npm install`) 所需的依附元件，以及執行小型本機主機伺服器來執行程式碼研究室 (執行 `node index.js`)。這些指令會在需要時顯示。

關於應用程式

WebRTC 使用用戶端 JavaScript API，但實際使用時也需要信號 (訊息) 伺服器，以及 STUN 和 TURN 伺服器。詳情請參閱[這篇文章](#)。

在這個步驟中，您將使用 Socket.IO Node.js 模組和 JavaScript 程式庫進行訊息傳送，建構簡單的 Node.js 信號伺服器。具備 Node.js 和 Socket.IO 的使用經驗會有幫助，但沒有也無妨，因為訊息元件非常簡單。

選擇合適的信號伺服器

本程式碼研究室使用 [Socket.IO](#) 做為信號伺服器。

Socket.IO 的設計可讓您輕鬆建構訊息交換服務，且由於內建「房間」概念，因此很適合用來瞭解 WebRTC 信號。

不過，對於正式版服務，有更好的替代方案。請參閱「[如何為下一個 WebRTC 專案選取信號通訊協定](#)」。

在本範例中，伺服器 (Node.js 應用程式) 是在 **index.js** 中實作，而伺服器上執行的用戶端 (網頁應用程式) 則是在 **index.html** 中實作。

這個步驟中的 Node.js 應用程式有兩項工作。

首先，它會做為訊息中繼：

```
socket.on('message', function (message) {  
  log('Got message: ', message);  
  socket.broadcast.emit('message', message);  
});
```

其次，它會管理 WebRTC 視訊通訊「室」：

```
if (numClients === 0) {  
  socket.join(room);  
  socket.emit('created', room, socket.id);  
} else if (numClients === 1) {  
  socket.join(room);  
  socket.emit('joined', room, socket.id);  
  io.sockets.in(room).emit('ready');  
} else { // max two clients  
  socket.emit('full', room);  
}
```

我們簡單的 WebRTC 應用程式最多允許兩位對等互連者共用一個會議室。

HTML 和 JavaScript

更新 **index.html**，使其看起來像這樣：


```
<!DOCTYPE html>
<html>

<head>

  <title>Realtime communication with WebRTC</title>

  <link rel="stylesheet" href="css/main.css" />

</head>

<body>

  <h1>Realtime communication with WebRTC</h1>

  <script src="/socket.io/socket.io.js"></script>
  <script src="js/main.js"></script>

</body>

</html>
```

在這個步驟中，您不會在頁面上看到任何內容，所有記錄都會寫入瀏覽器控制台。(如要在 Chrome 中查看控制台，請按下 Ctrl+Shift+J 鍵，或在 Mac 上按下 Command+Option+J 鍵)。

將 **js/main.js** 替換為下列內容：

```

'use strict';

var isInitiator;

window.room = prompt("Enter room name:");

var socket = io.connect();

if (room !== "") {
  console.log('Message from client: Asking to join room ' + room);
  socket.emit('create or join', room);
}

socket.on('created', function(room, clientId) {
  isInitiator = true;
});

socket.on('full', function(room) {
  console.log('Message from client: Room ' + room + ' is full :^(');
});

socket.on('ipaddr', function(ipaddr) {
  console.log('Message from client: Server IP address is ' + ipaddr);
});

socket.on('joined', function(room, clientId) {
  isInitiator = false;
});

socket.on('log', function(array) {
  console.log.apply(console, array);
});

```

設定 Socket.IO 在 Node.js 上執行

在 HTML 檔案中，您可能已發現自己使用的是 Socket.IO 檔案：

```
<script src="/socket.io/socket.io.js"></script>
```

在 **work** 目錄的頂層建立名為 **package.json** 的檔案，並加入以下內容：

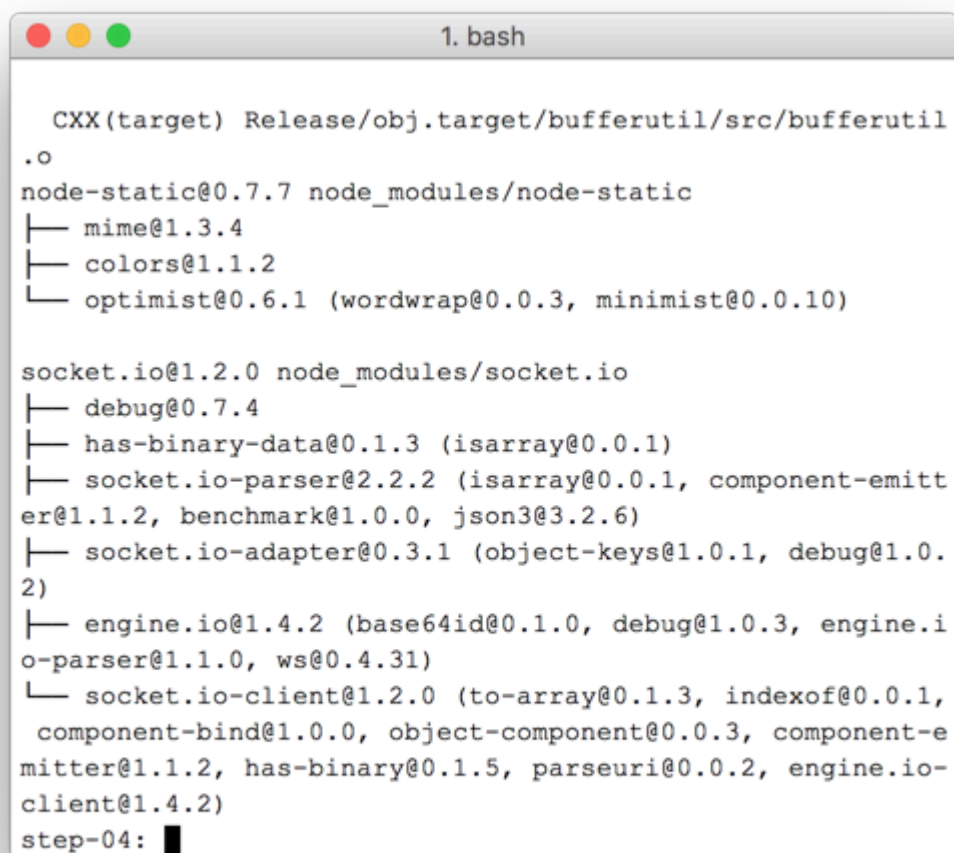
```
{
  "name": "webrtc-codelab",
  "version": "0.0.1",
  "description": "WebRTC codelab",
  "dependencies": {
    "node-static": "^0.7.10",
    "socket.io": "^1.2.0"
  }
}
```

這是應用程式資訊清單，可告知 Node Package Manager (`npm`) 要安裝哪些專案依附元件。

如要安裝依附元件 (例如 `/socket.io/socket.io.js`)，請在 **work** 目錄中，透過指令列終端機執行下列指令：

```
npm install
```

您應該會看到安裝記錄，結尾類似如下：



```
1. bash

CXX(target) Release/obj.target/bufferutil/src/bufferutil
.o
node-static@0.7.7 node_modules/node-static
├─ mime@1.3.4
├─ colors@1.1.2
└─ optimist@0.6.1 (wordwrap@0.0.3, minimist@0.0.10)

socket.io@1.2.0 node_modules/socket.io
├─ debug@0.7.4
├─ has-binary-data@0.1.3 (isarray@0.0.1)
├─ socket.io-parser@2.2.2 (isarray@0.0.1, component-emitter@1.1.2, benchmark@1.0.0, json3@3.2.6)
├─ socket.io-adapter@0.3.1 (object-keys@1.0.1, debug@1.0.2)
├─ engine.io@1.4.2 (base64id@0.1.0, debug@1.0.3, engine.io-parser@1.1.0, ws@0.4.31)
└─ socket.io-client@1.2.0 (to-array@0.1.3, indexof@0.0.1, component-bind@1.0.0, object-component@0.0.3, component-emitter@1.1.2, has-binary@0.1.5, parseuri@0.0.2, engine.io-client@1.4.2)
step-04: █
```

如您所見， `npm` 已安裝 `package.json` 中定義的依附元件。

在 **work** 目錄的頂層 (不在 `js` 目錄中) 建立新的 `index.js` 檔案，並加入下列程式碼：

```

'use strict';

var os = require('os');
var nodeStatic = require('node-static');
var http = require('http');
var socketIO = require('socket.io');

var fileServer = new(nodeStatic.Server)();
var app = http.createServer(function(req, res) {
  fileServer.serve(req, res);
}).listen(8080);

var io = socketIO.listen(app);
io.sockets.on('connection', function(socket) {

  // convenience function to log server messages on the client
  function log() {
    var array = ['Message from server:'];
    array.push.apply(array, arguments);
    socket.emit('log', array);
  }

  socket.on('message', function(message) {
    log('Client said: ', message);
    // for a real app, would be room-only (not broadcast)
    socket.broadcast.emit('message', message);
  });

  socket.on('create or join', function(room) {
    log('Received request to create or join room ' + room);

    var clientsInRoom = io.sockets.adapter.rooms[room];
    var numClients = clientsInRoom ? Object.keys(clientsInRoom.sockets).length : 0;

    log('Room ' + room + ' now has ' + numClients + ' client(s)');

    if (numClients === 0) {
      socket.join(room);
      log('Client ID ' + socket.id + ' created room ' + room);
      socket.emit('created', room, socket.id);
    } else if (numClients === 1) {
      log('Client ID ' + socket.id + ' joined room ' + room);
      io.sockets.in(room).emit('join', room);
      socket.join(room);
      socket.emit('joined', room, socket.id);
      io.sockets.in(room).emit('ready');
    } else { // max two clients
      socket.emit('full', room);
    }
  });
});

```

```
socket.on('ipaddr', function() {
  var ifaces = os.networkInterfaces();
  for (var dev in ifaces) {
    ifaces[dev].forEach(function(details) {
      if (details.family === 'IPv4' && details.address !== '127.0.0.1') {
        socket.emit('ipaddr', details.address);
      }
    });
  }
});

});
```

在指令列終端機中，於 **work** 目錄執行下列指令：

```
node index.js
```

在瀏覽器中開啟 **localhost:8080**。

每次開啟這個網址時，系統都會提示你輸入會議室名稱。如要加入同一個房間，每次請選擇相同的房間名稱，例如「foo」。

開啟新分頁，然後再次開啟 **localhost:8080**。選擇相同的房間名稱。

在第三個分頁或視窗中開啟 **localhost:8080**。再次選擇相同的房間名稱。

檢查每個分頁中的控制台，您應該會看到上述 JavaScript 的記錄。

獎勵分數

1. 有哪些替代訊息傳送機制？使用「純」WebSocket 時可能會遇到哪些問題？
2. 擴充這個應用程式時可能會遇到哪些問題？你能開發一種方法，測試數千或數百萬個同時發出的房間要求嗎？
3. 這個應用程式會使用 JavaScript 提示取得聊天室名稱。想辦法從網址取得聊天室名稱。舉例來說，*localhost:8080/foo* 會提供 `foo` 這個會議室名稱。

您學到的內容

在這個步驟中，您已瞭解如何：

- 使用 npm 安裝 package.json 中指定的專案依附元件
- 執行 Node.js 伺服器，提供靜態檔案。
- 使用 socket.io 在 Node.js 上設定訊息服務。
- 並用來建立「聊天室」和交換訊息。

這個步驟的完整版本位於「step-04」**step-04**資料夾中。

瞭解詳情

- [Socket.io chat-example 存放區](#)
- [真實世界中的 WebRTC：STUN、TURN 和信號](#)
- [WebRTC 中的「信號」一詞](#)

下一步

瞭解如何使用信號，讓兩位使用者建立對等互連。

步驟 8 · 約 5 分鐘

8. 合併對等互連和信號

課程內容

這個步驟會說明如何：

- 使用在 Node.js 上執行的 Socket.IO 執行 WebRTC 信號服務
- 使用該服務在對等互連裝置之間交換 WebRTC 中繼資料。

這個步驟的完整版本位於「step-05」**step-05**資料夾中。

取代 HTML 和 JavaScript

將 **index.html** 的內容替換為下列程式碼：

```
<!DOCTYPE html>
<html>

<head>

  <title>Realtime communication with WebRTC</title>

  <link rel="stylesheet" href="/css/main.css" />

</head>

<body>

  <h1>Realtime communication with WebRTC</h1>

  <div id="videos">
    <video id="localVideo" autoplay muted></video>
    <video id="remoteVideo" autoplay></video>
  </div>

  <script src="/socket.io/socket.io.js"></script>
  <script src="https://webrtc.github.io/adapter/adapter-latest.js"></script>
  <script src="js/main.js"></script>

</body>

</html>
```

將 **js/main.js** 替換為 **step-05/js/main.js** 的內容。

執行 Node.js 伺服器

如果您不是從工作目錄執行本程式碼研究室，可能需要為 **step-05** 資料夾或目前的工作資料夾安裝依附元件。在工作目錄中執行下列指令：

```
npm install
```

安裝完成後，如果 Node.js 伺服器未執行，請在 **work** 目錄中呼叫下列指令來啟動伺服器：

```
node index.js
```

請確認您使用的是上一個步驟中實作 Socket.IO 的 **index.js** 版本。如要進一步瞭解 Node 和 Socket IO，請參閱「設定訊號服務來交換訊息」一節。

在瀏覽器中開啟 **localhost:8080**。

在新分頁或視窗中再次開啟 **localhost:8080**。一個影片元素會顯示 `getUserMedia()` 的本機串流，另一個則會顯示透過 `RTCPeerconnection` 串流的「遠端」影片。

每次關閉用戶端分頁或視窗時，您都需要重新啟動 Node.js 伺服器。

在瀏覽器控制台中查看記錄。

獎勵點數

1. 這項應用程式僅支援一對一視訊通訊。如何變更設計，讓多人共用同一個視訊通訊室？
2. 這個範例已將會議室名稱 *foo* 硬式編碼。如何啟用其他房間名稱？
3. 使用者如何分享會議室名稱？嘗試建立替代方案，分享會議室名稱。
4. 如何變更應用程式

您學到的內容

您在此步驟中瞭解到如何：

- 使用在 Node.js 上執行的 Socket.IO 執行 WebRTC 信號服務。
- 使用該服務在對等互連裝置之間交換 WebRTC 中繼資料。

這個步驟的完整版本位於「step-05」**step-05**資料夾中。

訣竅

- 您可以透過 **chrome://webrtc-internals** 取得 WebRTC 統計資料和偵錯資料。
- 您可以使用 test.webrtc.org 檢查本機環境，並測試攝影機和麥克風。
- 如果快取發生奇怪的問題，請嘗試下列方法：
- 按住 Ctrl 鍵並點選「重新載入」按鈕，進行強制重新整理
- 重新啟動瀏覽器
- 透過指令列執行 `npm cache clean`。

下一步

瞭解如何拍照、取得圖片資料，以及在遠端對等互連裝置之間分享圖片。

9. 拍照並透過資料管道分享相片

課程內容

這個步驟將說明如何：

- 拍照並使用畫布元素取得相片資料。
- 與遠端使用者交換圖片資料。

這個步驟的完整版本位於「step-06」**step-06**資料夾中。

運作方式

您先前已瞭解如何使用 `RTCDataChannel` 交換簡訊。

這樣一來，您就能分享整個檔案，例如透過 `getUserMedia()` 拍攝的相片。

這個步驟的核心部分如下：

1. 建立資料管道。請注意，您不會在這個步驟中將任何媒體串流新增至對等連線。
2. 使用 `getUserMedia()` 擷取使用者的網路攝影機視訊串流：

```
var video = document.getElementById('video');

function grabWebCamVideo() {
  console.log('Getting user media (video) ...');
  navigator.mediaDevices.getUserMedia({
    video: true
  })
  .then(gotStream)
  .catch(function(e) {
    alert('getUserMedia() error: ' + e.name);
  });
}
```

3. 使用者點選「Snap」按鈕時，請從影片串流取得快照 (影片影格)，並顯示在 `canvas` 元素中：

```
var photo = document.getElementById('photo');
var photoContext = photo.getContext('2d');

function snapPhoto() {
  photoContext.drawImage(video, 0, 0, photo.width, photo.height);
  show(photo, sendBtn);
}
```

4. 使用者點選「傳送」按鈕時，請將圖片轉換為位元組，並透過資料管道傳送：

```

function sendPhoto() {
    // Split data channel message in chunks of this byte length.
    var CHUNK_LEN = 64000;
    var img = photoContext.getImageData(0, 0, photoContextW, photoContextH),
        len = img.data.byteLength,
        n = len / CHUNK_LEN | 0;

    console.log('Sending a total of ' + len + ' byte(s)');
    dataChannel.send(len);

    // split the photo and send in chunks of about 64KB
    for (var i = 0; i < n; i++) {
        var start = i * CHUNK_LEN,
            end = (i + 1) * CHUNK_LEN;
        console.log(start + ' - ' + (end - 1));
        dataChannel.send(img.data.subarray(start, end));
    }

    // send the reminder, if any
    if (len % CHUNK_LEN) {
        console.log('last ' + len % CHUNK_LEN + ' byte(s)');
        dataChannel.send(img.data.subarray(n * CHUNK_LEN));
    }
}

```

5. 接收端會將資料管道訊息位元組轉換回圖片，並向使用者顯示圖片：

```

function receiveDataChromeFactory() {
  var buf, count;

  return function onmessage(event) {
    if (typeof event.data === 'string') {
      buf = window.buf = new Uint8ClampedArray(parseInt(event.data));
      count = 0;
      console.log('Expecting a total of ' + buf.byteLength + ' bytes');
      return;
    }

    var data = new Uint8ClampedArray(event.data);
    buf.set(data, count);

    count += data.byteLength;
    console.log('count: ' + count);

    if (count === buf.byteLength) {
      // we're done: all data chunks have been received
      console.log('Done. Rendering photo.');
      renderPhoto(buf);
    }
  };
}

function renderPhoto(data) {
  var canvas = document.createElement('canvas');
  canvas.width = photoContextW;
  canvas.height = photoContextH;
  canvas.classList.add('incomingPhoto');
  // trail is the element holding the incoming images
  trail.insertBefore(canvas, trail.firstChild);

  var context = canvas.getContext('2d');
  var img = context.createImageData(photoContextW, photoContextH);
  img.data.set(data);
  context.putImageData(img, 0, 0);
}

```

取得程式碼

將 **work** 資料夾的內容替換為 **step-06** 的內容。 **work** 中的 **index.html** 檔案現在應如下所示：

```

<!DOCTYPE html>
<html>

<head>

  <title>Realtime communication with WebRTC</title>

  <link rel="stylesheet" href="/css/main.css" />

</head>

<body>

  <h1>Realtime communication with WebRTC</h1>

  <h2>
    <span>Room URL: </span><span id="url">...</span>
  </h2>

  <div id="videoCanvas">
    <video id="camera" autoplay></video>
    <canvas id="photo"></canvas>
  </div>

  <div id="buttons">
    <button id="snap">Snap</button><span> then </span><button id="send">Send</button>
    <span> or </span>
    <button id="snapAndSend">Snap & Send</button>
  </div>

  <div id="incoming">
    <h2>Incoming photos</h2>
    <div id="trail"></div>
  </div>

  <script src="/socket.io/socket.io.js"></script>
  <script src="https://webrtc.github.io/adapters/adapters-latest.js"></script>
  <script src="js/main.js"></script>

</body>

</html>

```

如果您不是從工作目錄進行本程式碼研究室，可能需要為 **step-06** 資料夾或目前的工作資料夾安裝依附元件。只要從工作目錄執行下列指令即可：

```
npm install
```

安裝完成後，如果 Node.js 伺服器未執行，請從 **work** 目錄呼叫下列指令來啟動伺服器：

```
node index.js
```

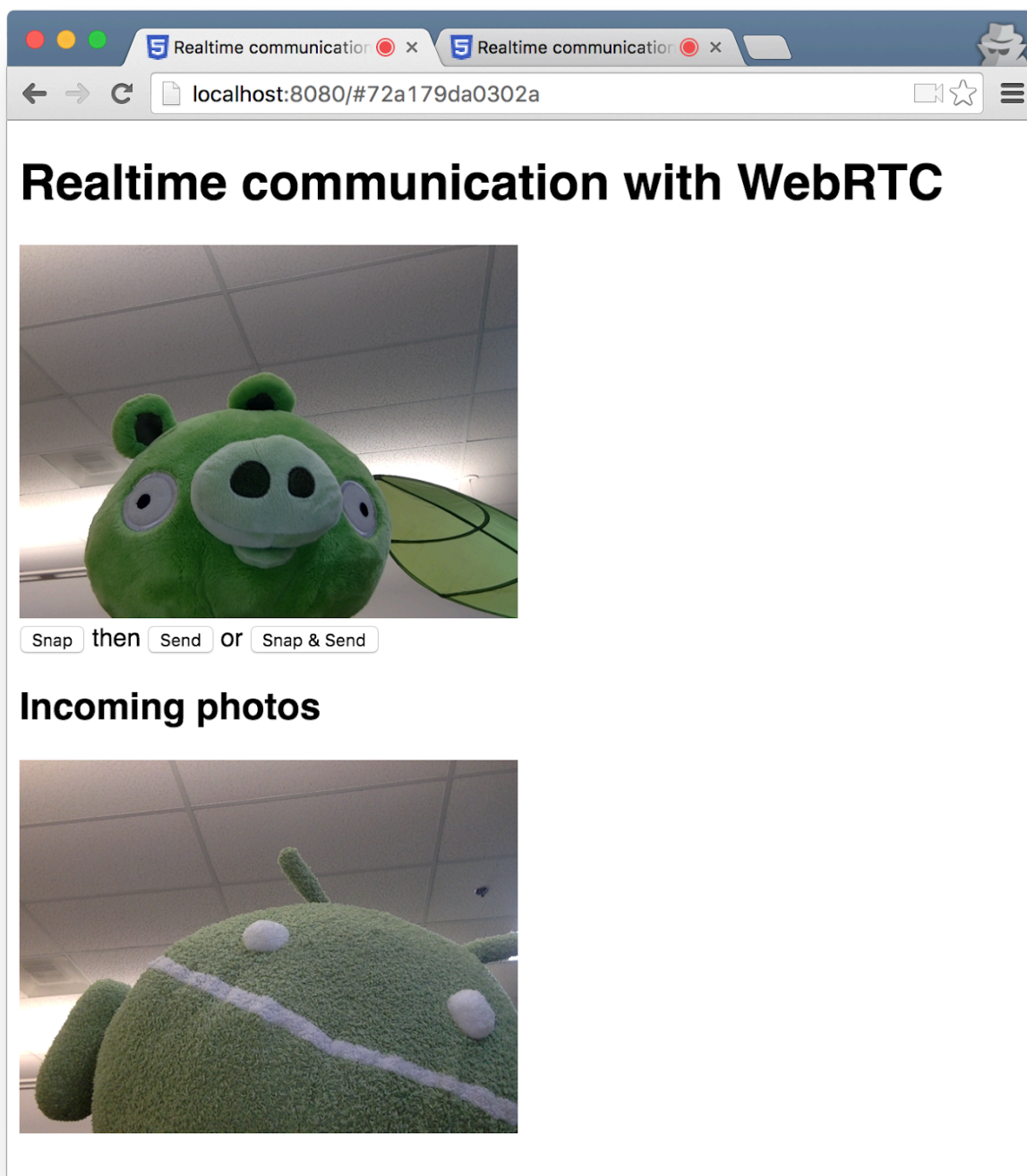
請確認您使用的是實作 Socket.IO 的 **index.js** 版本，並記得在變更後重新啟動 Node.js 伺服器。如要進一步瞭解 Node 和 Socket IO，請參閱「設定訊號服務來交換訊息」一節。

如有必要，請按一下「允許」按鈕，允許應用程式使用網路攝影機。

應用程式會建立隨機的房間 ID，並將該 ID 新增至網址。在新瀏覽器分頁或視窗中開啟網址列中的網址。

按一下「Snap & Send」(拍攝並傳送) 按鈕，然後查看另一個分頁頁面底部的「Incoming」(傳入) 區域。應用程式會將相片從一個分頁轉移到另一個分頁。

畫面應如下所示：



獎勵分數

1. 如何變更程式碼，以便共用任何檔案類型？

瞭解詳情

- [MediaStream Image Capture API](#)：用於圖片拍攝及控制攝影機的 API，即將在您附近的瀏覽器推出！
- MediaRecorder API，用於錄製音訊和影片：[示範](#)、[說明文件](#)。

您學到的內容

- 如何使用 Canvas 元素拍照並取得相片資料。
- 如何與遠端使用者交換資料。

這個步驟的完整版本位於「step-06」**step-06**資料夾中。

步驟 10 · 約 1 分鐘

10. 恭喜

您已建構應用程式，可進行即時視訊串流和資料交換！

您學到的內容

在本程式碼研究室中，您瞭解如何：

- 從網路攝影機取得影片。
- 使用 `RTCPeerConnection` 串流播放影片。
- 使用 `RTCDataChannel` 串流資料。
- 設定信號服務來交換訊息。
- 合併對等互連和信號。
- 拍照並透過資料管道分享相片。

後續步驟

- 查看標準 WebRTC 即時通訊應用程式 AppRTC 的程式碼和架構：[應用程式](#)、[程式碼](#)。
- 請前往 [github.com/webrtc/samples](https://github.com/webRTC/samples) 試用[即時示範](#)。

瞭解詳情

- 如要開始使用 WebRTC，請前往 webrtc.org 取得各種資源。
-